

Experiment-01

Aim:

A. Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are: - “Low Salary”: All monthly salaries strictly less than \$20,000. - “Average Salary”: All monthly salaries in the inclusive range [\$20,000, \$50,000]. - “High Salary”: All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

B. Design an Employee Management System database architecture by implementing the following operations sequentially: - Create and populate structural tracking tables for Departments, Employees, and Salaries. - Query the system using multi-table Joins to pull clear department breakdowns. - List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

C. A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`. Every customer must choose exactly 3 different toppings to prepare a pizza. Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost. Requirements: - Every pizza must contain exactly 3 different toppings. - The same topping cannot appear more than once in a pizza. - Different arrangements of the same toppings should be considered the same pizza. - Display the topping names separated by commas. - Display the total ingredient cost of each pizza. - Sort the output: first by highest total cost, then alphabetically if tied.

D. Amazon wants to identify returning customers who made another purchase shortly after their first purchase. Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase. Requirements: - Compare purchases made by the same user only. - Ignore comparisons where the same transaction is matched with itself. - The second purchase must occur after the first purchase. - Same-day purchases must not be considered. - The second purchase must occur within 7 days of the first purchase. - Display only the unique User IDs that satisfy the above conditions. - Sort the output in ascending order of `user_id`.

Tools / Platform Used:

- Windows
- PostgreSQL
- pgAdmin

Theory:

A JOIN is an SQL operation used to combine rows from two or more tables based on a related column between them. Joins help retrieve meaningful information stored across multiple tables while maintaining database normalization.

The most common types of joins are:

- INNER JOIN - returns only the rows that have matching values in both tables. If a row has no corresponding match in the other table, it is not included in the result.
- LEFT OUTER JOIN - returns all rows from the left table and the matching rows from the right table. If no match exists, NULL values are returned for the right table columns.
- RIGHT OUTER JOIN - returns all rows from the right table and matching rows from the left table. If no matching record exists in the left table, NULL values are displayed.
- FULL OUTER JOIN - returns all records from both tables. Matching records are combined, while non-matching rows from either table contain NULL values.
- CROSS JOIN - returns the Cartesian Product of two tables. Every row from the first table is combined with every row from the second table.
- SELF JOIN - joins a table with itself using aliases. It is useful when rows within the same table have relationships.
- LEFT EXCLUSIVE JOIN - returns only those rows from the left table that have no matching rows in the right table.
- RIGHT EXCLUSIVE JOIN - returns only those rows from the right table that have no matching rows in the left table.

Code and Output:

A.

```
CREATE TABLE bank_accounts (  
    account_id INT PRIMARY KEY,  
    monthly_salary INT NOT NULL  
);
```

```
INSERT INTO bank_accounts (account_id, monthly_salary) VALUES  
(101, 15000),  
(102, 32000),  
(103, 61500),  
(104, 20000),  
(105, 50000),  
(106, 9800),  
(107, 75200);
```

```

1 CREATE TABLE bank_accounts (
2     account_id INT PRIMARY KEY,
3     monthly_salary INT NOT NULL
4 );
5
6 INSERT INTO bank_accounts (account_id, monthly_salary) VALUES
7     (101, 15000),
8     (102, 32000),
9     (103, 61500),
10    (104, 20000),
11    (105, 50000),
12    (106, 9800),
13    (107, 75200);
14

```

STDIN

Input for the program (Optional)

Output:

CREATE TABLE
INSERT 0 7

```

SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count
FROM bank_accounts WHERE monthly_salary < 20000

```

UNION ALL

```

SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count
FROM bank_accounts WHERE monthly_salary >= 20000 AND monthly_salary <= 50000

```

UNION ALL

```

SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count
FROM bank_accounts WHERE monthly_salary > 50000;

```

```

1 CREATE TABLE bank_accounts (
2     account_id INT PRIMARY KEY,
3     monthly_salary INT NOT NULL
4 );
5
6 INSERT INTO bank_accounts (account_id, monthly_salary) VALUES
7     (101, 15000),
8     (102, 32000),
9     (103, 61500),
10    (104, 20000),
11    (105, 50000),
12    (106, 9800),
13    (107, 75200);
14
15 SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count
16 FROM bank_accounts WHERE monthly_salary < 20000
17
18 UNION ALL
19
20 SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count
21 FROM bank_accounts WHERE monthly_salary >= 20000 AND monthly_salary <= 50000
22
23 UNION ALL
24
25 SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count
26 FROM bank_accounts WHERE monthly_salary > 50000;
27

```

STDIN

Input for the program (Optional)

Output:

```

CREATE TABLE
INSERT 0 7

```

category	accounts_count
Low Salary	2
Average Salary	3
High Salary	2

(3 rows)

B.

```
CREATE TABLE dept (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE emp (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES dept(dept_id)
);
```

```
CREATE TABLE emp_salary (
    emp_id INT PRIMARY KEY REFERENCES emp(emp_id) ON DELETE CASCADE,
    salary INT NOT NULL
);
```

```
INSERT INTO dept (dept_id, dept_name) VALUES
(1, 'Finance'), (2, 'Engineering'), (3, 'Marketing');
```

```
INSERT INTO emp (emp_id, emp_name, dept_id) VALUES
(1, 'Rhyddham', 2), (2, 'Aarav', 1), (3, 'Priya', 2), (4, 'Sana', 3), (5, 'Kabir', 1);
```

```
INSERT INTO emp_salary (emp_id, salary) VALUES
(1, 72000), (2, 38000), (3, 54000), (4, 29000), (5, 61000);
```

```
1 CREATE TABLE dept (
2     dept_id INT PRIMARY KEY,
3     dept_name VARCHAR(50) NOT NULL
4 );
5
6 CREATE TABLE emp (
7     emp_id INT PRIMARY KEY,
8     emp_name VARCHAR(50) NOT NULL,
9     dept_id INT REFERENCES dept(dept_id)
10 );
11
12 CREATE TABLE emp_salary (
13     emp_id INT PRIMARY KEY REFERENCES emp(emp_id) ON DELETE CASCADE,
14     salary INT NOT NULL
15 );
16
17 INSERT INTO dept (dept_id, dept_name) VALUES
18 (1, 'Finance'), (2, 'Engineering'), (3, 'Marketing');
19
20 INSERT INTO emp (emp_id, emp_name, dept_id) VALUES
21 (1, 'Rhyddham', 2), (2, 'Aarav', 1), (3, 'Priya', 2), (4, 'Sana', 3), (5, 'Kabir', 1);
22
23 INSERT INTO emp_salary (emp_id, salary) VALUES
24 (1, 72000), (2, 38000), (3, 54000), (4, 29000), (5, 61000);
25
```

STDIN

Input for the program (Optional)

Output:

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 5
INSERT 0 5
```

```
SELECT e.emp_name, d.dept_name, s.salary
FROM emp AS e
JOIN dept AS d ON e.dept_id = d.dept_id
JOIN emp_salary AS s ON e.emp_id = s.emp_id;
```

```

1 CREATE TABLE dept (
2   dept_id INT PRIMARY KEY,
3   dept_name VARCHAR(50) NOT NULL
4 );
5
6 CREATE TABLE emp (
7   emp_id INT PRIMARY KEY,
8   emp_name VARCHAR(50) NOT NULL,
9   dept_id INT REFERENCES dept(dept_id)
10 );
11
12 CREATE TABLE emp_salary (
13   emp_id INT PRIMARY KEY REFERENCES emp(emp_id) ON DELETE CASCADE,
14   salary INT NOT NULL
15 );
16
17 INSERT INTO dept (dept_id, dept_name) VALUES
18 (1, 'Finance'), (2, 'Engineering'), (3, 'Marketing');
19
20 INSERT INTO emp (emp_id, emp_name, dept_id) VALUES
21 (1, 'Rhyddham', 2), (2, 'Aarav', 1), (3, 'Priya', 2), (4, 'Sana', 3), (5, 'Kabir', 1);
22
23 INSERT INTO emp_salary (emp_id, salary) VALUES
24 (1, 72000), (2, 38000), (3, 54000), (4, 29000), (5, 61000);
25 SELECT e.emp_name, d.dept_name, s.salary
26 FROM emp AS e
27 JOIN dept AS d ON e.dept_id = d.dept_id
28 JOIN emp_salary AS s ON e.emp_id = s.emp_id;

```

STDIN
Input for the program (Optional)

Output:

```

CREATE TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 5
INSERT 0 5
emp_name | dept_name | salary
-----+-----+-----
Rhyddham | Engineering | 72000
Aarav    | Finance    | 38000
Priya    | Engineering | 54000
Sana     | Marketing  | 29000
Kabir    | Finance    | 61000
(5 rows)

```

```

SELECT e.emp_name, s.salary
FROM emp AS e
JOIN emp_salary AS s ON e.emp_id = s.emp_id
WHERE s.salary > (SELECT AVG(salary) FROM emp_salary);

```

```

1 CREATE TABLE dept (
2   dept_id INT PRIMARY KEY,
3   dept_name VARCHAR(50) NOT NULL
4 );
5
6 CREATE TABLE emp (
7   emp_id INT PRIMARY KEY,
8   emp_name VARCHAR(50) NOT NULL,
9   dept_id INT REFERENCES dept(dept_id)
10 );
11
12 CREATE TABLE emp_salary (
13   emp_id INT PRIMARY KEY REFERENCES emp(emp_id) ON DELETE CASCADE,
14   salary INT NOT NULL
15 );
16
17 INSERT INTO dept (dept_id, dept_name) VALUES
18 (1, 'Finance'), (2, 'Engineering'), (3, 'Marketing');
19
20 INSERT INTO emp (emp_id, emp_name, dept_id) VALUES
21 (1, 'Rhyddham', 2), (2, 'Aarav', 1), (3, 'Priya', 2), (4, 'Sana', 3), (5, 'Kabir', 1);
22
23 INSERT INTO emp_salary (emp_id, salary) VALUES
24 (1, 72000), (2, 38000), (3, 54000), (4, 29000), (5, 61000);
25 SELECT e.emp_name, d.dept_name, s.salary
26 FROM emp AS e
27 JOIN dept AS d ON e.dept_id = d.dept_id
28 JOIN emp_salary AS s ON e.emp_id = s.emp_id;
29 SELECT e.emp_name, s.salary
30 FROM emp AS e
31 JOIN emp_salary AS s ON e.emp_id = s.emp_id
32 WHERE s.salary > (SELECT AVG(salary) FROM emp_salary);
33

```

STDIN
Input for the program (Optional)

Output:

```

CREATE TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 5
INSERT 0 5
emp_name | dept_name | salary
-----+-----+-----
Rhyddham | Engineering | 72000
Aarav    | Finance    | 38000
Priya    | Engineering | 54000
Sana     | Marketing  | 29000
Kabir    | Finance    | 61000
(5 rows)

emp_name | salary
-----+-----
Rhyddham | 72000
Priya    | 54000
Kabir    | 61000
(3 rows)

```

C.

```

CREATE TABLE pizza_toppings (
  topping_id INT PRIMARY KEY,
  topping_name VARCHAR(50) NOT NULL,
  cost NUMERIC NOT NULL
);

```

```

INSERT INTO pizza_toppings (topping_id, topping_name, cost) VALUES
(1, 'Mushroom', 30),
(2, 'Onion', 20),
(3, 'Paneer', 45),

```

```
(4, 'Corn', 25),
(5, 'Capsicum', 22);
```

```
1 CREATE TABLE pizza_toppings (
2     topping_id INT PRIMARY KEY,
3     topping_name VARCHAR(50) NOT NULL,
4     cost NUMERIC NOT NULL
5 );
6
7 INSERT INTO pizza_toppings (topping_id, topping_name, cost) VALUES
8 (1, 'Mushroom', 30),
9 (2, 'Onion', 20),
10 (3, 'Paneer', 45),
11 (4, 'Corn', 25),
12 (5, 'Capsicum', 22);
13
```

STDIN
Input for the program (Optional)

Output:
CREATE TABLE
INSERT 0 5

```
SELECT
    STRING_AGG(t.topping_name, ', ' ORDER BY t.topping_name) AS pizza,
    SUM(t.cost) AS total_cost
FROM pizza_toppings t1
JOIN pizza_toppings t2 ON t1.topping_id < t2.topping_id
JOIN pizza_toppings t3 ON t2.topping_id < t3.topping_id
JOIN pizza_toppings t ON t.topping_id IN (t1.topping_id, t2.topping_id, t3.topping_id)
GROUP BY t1.topping_id, t2.topping_id, t3.topping_id
ORDER BY total_cost DESC, pizza ASC;
```

```
CREATE TABLE pizza_toppings (
    topping_id INT PRIMARY KEY,
    topping_name VARCHAR(50) NOT NULL,
    cost NUMERIC NOT NULL
);

INSERT INTO pizza_toppings (topping_id, topping_name, cost) VALUES
(1, 'Mushroom', 30),
(2, 'Onion', 20),
(3, 'Paneer', 45),
(4, 'Corn', 25),
(5, 'Capsicum', 22);
SELECT
    STRING_AGG(t.topping_name, ', ' ORDER BY t.topping_name) AS pizza,
    SUM(t.cost) AS total_cost
FROM pizza_toppings t1
JOIN pizza_toppings t2 ON t1.topping_id < t2.topping_id
JOIN pizza_toppings t3 ON t2.topping_id < t3.topping_id
JOIN pizza_toppings t ON t.topping_id IN (t1.topping_id, t2.topping_id, t3.topping_id)
GROUP BY t1.topping_id, t2.topping_id, t3.topping_id
ORDER BY total_cost DESC, pizza ASC;
```

STDIN
Input for the program (Optional)

Output:
CREATE TABLE
INSERT 0 5

pizza	total_cost
Corn, Mushroom, Paneer	100
Capsicum, Mushroom, Paneer	97
Mushroom, Onion, Paneer	95
Capsicum, Corn, Paneer	92
Corn, Onion, Paneer	90
Capsicum, Onion, Paneer	87
Capsicum, Corn, Mushroom	77
Corn, Mushroom, Onion	75
Capsicum, Mushroom, Onion	72
Capsicum, Corn, Onion	67

(10 rows)

D.

```
CREATE TABLE purchases (
    txn_id INT PRIMARY KEY,
    user_id INT NOT NULL,
    purchase_date DATE NOT NULL
);
```

```
INSERT INTO purchases (txn_id, user_id, purchase_date) VALUES
(1, 501, '2026-01-02'),
(2, 501, '2026-01-06'),
(3, 502, '2026-01-03'),
```

```
(4, 502, '2026-01-15'),
(5, 503, '2026-01-05'),
(6, 503, '2026-01-05'),
(7, 504, '2026-01-01'),
(8, 504, '2026-01-04');
```

```
1 CREATE TABLE purchases (
2     txn_id INT PRIMARY KEY,
3     user_id INT NOT NULL,
4     purchase_date DATE NOT NULL
5 );
6
7 INSERT INTO purchases (txn_id, user_id, purchase_date) VALUES
8 (1, 501, '2026-01-02'),
9 (2, 501, '2026-01-06'),
10 (3, 502, '2026-01-03'),
11 (4, 502, '2026-01-15'),
12 (5, 503, '2026-01-05'),
13 (6, 503, '2026-01-05'),
14 (7, 504, '2026-01-01'),
15 (8, 504, '2026-01-04');
```

STDIN
Input for the program (Optional)

Output:
CREATE TABLE
INSERT 0 8

```
SELECT DISTINCT p1.user_id
FROM purchases p1
JOIN purchases p2
    ON p1.user_id = p2.user_id
    AND p1.txn_id <> p2.txn_id
    AND p2.purchase_date > p1.purchase_date
    AND p2.purchase_date <= p1.purchase_date + INTERVAL '7 days'
ORDER BY p1.user_id;
```

```
1 CREATE TABLE purchases (
2     txn_id INT PRIMARY KEY,
3     user_id INT NOT NULL,
4     purchase_date DATE NOT NULL
5 );
6
7 INSERT INTO purchases (txn_id, user_id, purchase_date) VALUES
8 (1, 501, '2026-01-02'),
9 (2, 501, '2026-01-06'),
10 (3, 502, '2026-01-03'),
11 (4, 502, '2026-01-15'),
12 (5, 503, '2026-01-05'),
13 (6, 503, '2026-01-05'),
14 (7, 504, '2026-01-01'),
15 (8, 504, '2026-01-04');
```

```
1
2
3
4
5
6
7
8 SELECT DISTINCT p1.user_id
9 FROM purchases p1
10 JOIN purchases p2
11     ON p1.user_id = p2.user_id
12     AND p1.txn_id <> p2.txn_id
13     AND p2.purchase_date > p1.purchase_date
14     AND p2.purchase_date <= p1.purchase_date + INTERVAL '7 days'
15 ORDER BY p1.user_id;
```

STDIN
Input for the program (Optional)

Output:
CREATE TABLE
INSERT 0 8
user_id

501
504
(2 rows)

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL

- Got hands-on experience on working with JOINS in PostgreSQL.
- Understood the concept of JOINS.